Proceedings of the 2026 Winter Simulation Conference
V. Ramamohan, A. Djanatliev, M. Fakhimi, C. Krejci, C. Ruiz Martin, B. S. Onggo, and N. Mustafee, eds.

DISTRIBUTED PROXIMAL POLICY OPTIMIZATION FOR WAFER-LOT DISPATCHING IN SEMICONDUCTOR MANUFACTURING DIGITAL TWINS WITH 2D3PO

Anonymous

ABSTRACT

This paper presents a distributed Proximal Policy Optimization (PPO) framework and software toolkit for accelerating wafer-lot dispatching policy learning in semiconductor manufacturing digital twins. The new framework, named 2D3PO, combines Ray-based distributed rollout collection with PyTorch Distributed Data Parallel synchronization while preserving compatibility with Stable-Baselines3. Experiments were conducted in FabSim, a micro-discrete event simulation environment for high-mix, low-volume semiconductor scheduling and dispatching, to evaluate whether scaling rollout collection can reduce wall-clock training time for PPO-based dispatching policies without sacrificing final operational performance as measured by makespan. Results show substantial runtime improvement as the distributed configuration increases, with the largest configuration completing training 3.7 times faster than the 16CPU baseline. Final makespan comparisons found no statistically significant differences relative to the baseline under Dunnett tests. These results show that distributed PPO can make semiconductor digital twin training markedly more efficient and more practical for iterative wafer-lot dispatching studies under the tested conditions.

1 INTRODUCTION

Semiconductor wafer fabrication is a complex production setting in which wafer-lot dispatching and short-term scheduling decisions must be made under high variability, reentrant flows, long process routes, and limited machine capacity (Pfund et al. 2006; Sarin et al. 2011). These characteristics make semiconductor operations a strong candidate for digital-twin-based decision support, where simulation can be used to train and evaluate adaptive control policies before deployment (Ouahabi et al. 2024; Kuo et al. 2025). In this setting, *reinforcement learning (RL)* offers a promising approach for learning dispatching policies directly from repeated interaction with a simulation environment rather than relying only on fixed dispatching rules.

This paper studies distributed *Proximal Policy Optimization (PPO)* training with *FabSim*, a micro-discrete event simulation environment for high-mix, low-volume (HMLV) semiconductor wafer-lot scheduling and dispatching (Nsiye et al. 2024; Mondesire et al. 2025). Specifically, it introduces 2D3PO, a distributed PPO framework and software toolkit for training wafer-lot dispatching policies in FabSim. FabSim serves as the digital twin environment in which an RL agent learns dispatching decisions through simulation, while policy quality is evaluated using the resulting makespan. The goal of this work is not to propose a new PPO variant, but to make PPO-based digital twin training more computationally practical for semiconductor dispatching through a new distributed RL training framework and toolkit.

With traditional, centralized training for a dispatching agent, single-node PPO training becomes slow as the scale of the problem and the available training budget increase. This poor scaling is due to two main challenges: 1) simulations that virtualize fab conditions are traditionally CPU-dependent and do not benefit from GPU parallelization in the same way as many other machine learning workloads and 2) wafer-lot dispatching can be viewed as a form of the Job Shop Scheduling Problem (JSSP), a classically hard scheduling problem whose difficulty grows rapidly with problem size. Both challenges increase agent training time as fab conditions grow in complexity, including the number of wafer lots, process flows and process steps, and tools (machines). To reduce this bottleneck, rollout collection is distributed across compute nodes using *Ray*, and synchronized policy optimization is performed with *PyTorch Distributed Data Parallel (DDP)* while preserving compatibility with *Stable-Baselines3 (SB3)* PPO implementations (Moritz et al. 2018; Li et al. 2020; Raffin et al. 2021).

The central question of this work is whether distributed rollout collection can make PPO-based digital twin training more practical for semiconductor wafer-lot dispatching while maintaining competitive policy performance. This distributed framework was evaluated by training dispatching policies under increasing distributed configurations and comparing both wall-clock runtime and makespan-based performance. Runtime matters because faster training supports more efficient digital twin retraining and scenario iteration, while dispatching performance matters because reduced training time is only useful if the learned policy remains operationally effective.

The contribution of this work is threefold. First, it presents an RL workflow for wafer-lot dispatching within semiconductor digital twin training environments using PPO. Second, it introduces 2D3PO, a Stable-Baselines3-compatible distributed training framework and software toolkit that accelerates policy learning by parallelizing rollout collection across multiple nodes while maintaining a modular implementation based on SB3, Ray, and DDP. Third, it provides an empirical study of how distributed training affects both runtime and makespan-based dispatching performance in a semiconductor-inspired simulation setting.

2 BACKGROUND

Semiconductor wafer fabrication is one of the most complex scheduling environments in manufacturing. The scheduling task in such systems can be viewed as a form of the JSSP, in which jobs must be assigned to machines across ordered processing steps while satisfying resource and precedence constraints and optimizing a performance objective such as makespan (Pfund et al. 2006; Gupta and Sivakumar 2006). In semiconductor fabs, this problem is especially difficult because wafer lots move through long process routes, revisit equipment groups multiple times, compete for limited tool capacity, and are subject to batching, setup effects, reentrant flow, and changing fab conditions (Sarin et al. 2011). In this paper, *dispatching* refers to the local wafer-lot action selected at a decision point, while *scheduling* is used more broadly for the overall production-control problem.

The classical JSSP is computationally hard. More precisely, its decision form is NP-complete, meaning that no known method can guarantee efficient optimal solutions for all possible instances as problem size grows (Garey et al. 1976). In practical terms, this means that exact scheduling becomes increasingly difficult as the number of jobs, machines, and constraints increases. These complexity challenges are amplified in semiconductor settings, where operational decisions must also respond to dynamic system conditions and downstream interactions across the production line. Consequently, short-term dispatching and scheduling decisions are difficult to optimize using fixed rules alone, which motivates adaptive decision methods that can respond to evolving shop-floor conditions.

2.1 Digital Twins for Semiconductor Operations

Digital twins have emerged as an important approach for representing manufacturing systems in a form that supports experimentation, monitoring, and decision support. In production settings, a digital twin is commonly used to reflect the behavior of a physical system closely enough to evaluate alternative policies, analyze operational tradeoffs, and support adaptive control without disrupting live production (Ouahabi et al. 2024; Schwede and Fischer 2024; Wooley et al. 2025). For scheduling problems, this capability is especially valuable because candidate control strategies can be tested repeatedly under varied system conditions before they are deployed.

This perspective is highly relevant to semiconductor operations. Recent work has shown that digital twin frameworks can support production planning and scheduling in wafer fabrication, including settings with uncertainty and rapidly changing operating conditions (Kuo et al. 2025). In parallel, simulation environments have been developed specifically to support production scheduling research in manufacturing digital twins (Nsiye et al. 2024). Within this paper, FabSim plays this role as the simulation environment within a digital-twin workflow. FabSim is a micro-discrete event simulation environment designed for machine learning based dynamic job shop scheduling and provides a simulation-based setting in which semiconductor-

inspired scheduling and dispatching policies can be trained and evaluated repeatedly (Mondesire et al. 2025). This makes it an appropriate platform for studying reinforcement learning as a decision method for digital-twin-driven semiconductor operations.

2.2 Reinforcement Learning for Scheduling and Dispatching

RL is well suited to sequential decision problems in which the quality of an action depends on both the current system state and its downstream consequences. In scheduling and dispatching contexts, an RL agent observes a representation of the production state, selects a control action, and receives feedback through a performance signal tied to the operational objective. Over repeated interaction, the agent learns a policy that maps observed system conditions to actions that improve long-horizon performance. This differs from fixed dispatching rules because the policy is learned from experience rather than prescribed in advance.

RL has been applied increasingly to scheduling and optimization problems in manufacturing and industrial engineering (Waschneck et al. 2018; Chung et al. 2025). Within semiconductor manufacturing, recent work has explored RL for fab scheduling, dispatching, and production planning, including self-supervised and reinforcement learning approaches for semiconductor fab scheduling, digital twin frameworks for resilient wafer fabrication planning, and studies focused on validation and deployment of RL based dispatching solutions in semiconductor settings (Tassel et al. 2023; Kuo et al. 2025; Stöckermann et al. 2025). This literature suggests that RL is a promising direction for semiconductor control, but it also highlights practical challenges related to evaluation rigor, deployment readiness, and the computational cost of policy training (Stöckermann et al. 2025; Yedidsion et al. 2025). These concerns are particularly important when policies must be trained and compared across multiple digital twin scenarios rather than a single static environment.

2.3 PPO and the Cost of Policy Training

Among policy gradient methods, PPO is widely used because it provides a stable and practical balance between optimization performance and implementation simplicity (Schulman et al. 2017). PPO collects rollout data under the current policy and then performs multiple optimization passes over that batch using a clipped surrogate objective that discourages overly large policy updates. In practice, PPO is often paired with *generalized advantage estimation (GAE)* and other stabilization techniques, making it a strong baseline for continuous and discrete control tasks alike (Schulman et al. 2015; Andrychowicz et al. 2021; Raffin et al. 2021).

Despite these advantages, PPO is computationally expensive. Because it is an on-policy method, new rollout data must be generated repeatedly as training progresses, and each batch is used for multiple gradient updates before the next data collection phase begins (Schulman et al. 2017). In simulation-based scheduling environments, this means that large numbers of environment interactions are required to obtain a competitive policy. The resulting wall-clock cost can slow experimentation, limit hyperparameter search, and reduce the practical value of RL within digital twin workflows where many policy variants may need to be trained and compared (Engstrom et al. 2020; Andrychowicz et al. 2021). For semiconductor dispatching studies, where policies may need to be retrained across different fab conditions and operating assumptions, the training bottleneck becomes a central practical concern.

2.4 Distributed PPO Training

Distributed training offers a natural way to reduce PPO wall-clock cost by parallelizing rollout generation and synchronizing policy updates across multiple workers. More broadly, data-parallel learning has become a standard mechanism for scaling machine learning workloads: model replicas process different subsets of data, gradients are aggregated through collective communication, and the resulting update is applied consistently across workers (Shallue et al. 2019; Ben-Nun and Hoefler 2019; Li et al. 2020). In reinforcement

learning, this idea extends to distributed rollout collection, where multiple workers interact with environment instances concurrently to increase data throughput.

For the present work, the important issue is not simply whether distributed RL can scale, but whether it can do so in a way that preserves a controlled and interpretable PPO training procedure. Prior studies have shown that RL results can be sensitive to implementation details, evaluation choices, and other seemingly small experimental decisions (Henderson et al. 2018; Agarwal et al. 2021). This makes modularity and algorithmic consistency especially important. A distributed PPO workflow built around reliable components such as SB3 for policy optimization, Ray for distributed environment execution, and PyTorch DDP for gradient synchronization provides a practical way to introduce distribution without rewriting PPO from scratch (Moritz et al. 2018; Raffin et al. 2021; Li et al. 2020). Such a design is well matched to digital twin experimentation because it supports scaling while retaining a familiar and reproducible learning implementation.

Within distributed PPO, however, batch-dependent preprocessing steps still require care. Advantage estimation and normalization are commonly used to stabilize policy gradient learning (Sutton et al. 1999; Schulman et al. 2015; Greensmith et al. 2004). In a distributed setting, statistics used for normalization may be computed locally on each worker or aggregated globally across workers before normalization is applied. Since these choices affect gradient scaling, they can influence how closely the distributed update matches the interpretation of PPO as operating on a single larger batch. In this paper, globally normalized advantages are treated as a design choice that helps maintain consistency across workers within the distributed training procedure, while the main focus remains the practical value of distributed rollout collection for FabSim training.

2.5 Research Gap and Study Objective

Prior work therefore establishes three key points. First, semiconductor scheduling and dispatching are difficult sequential decision problems that are not always well served by fixed heuristics alone (Pfund et al. 2006; Sarin et al. 2011). Second, digital twins provide a useful simulation-based setting for evaluating and training adaptive control policies in manufacturing and semiconductor operations (Ouahabi et al. 2024; Kuo et al. 2025; Mondesire et al. 2025). Third, RL and PPO in particular offer a practical way to learn such policies, but the training process is expensive enough to limit iterative experimentation (Schulman et al. 2017; Andrychowicz et al. 2021). Recent semiconductor RL work has focused primarily on policy quality, validation, or deployment readiness, whereas the training-system bottleneck has received less direct attention (Stöckermann et al. 2025; Yedidsion et al. 2025). What remains less clear is whether distributed PPO training can make RL-based digital twin workflows more practical for semiconductor dispatching without undermining the quality of the learned policy.

This gap motivates the present study. Using FabSim as a semiconductor-inspired micro-discrete event simulation environment that follows the *Gymnasium (Gym)* interface, this work investigates whether distributed rollout collection can reduce PPO training time while preserving useful scheduling performance as measured by makespan (Towers et al. 2025; Nsiye et al. 2024; Mondesire et al. 2025). More specifically, the study examines a distributed training framework that combines Ray-based environment parallelism with DDP-based synchronized optimization while maintaining compatibility with SB3 PPO. The central research question is: *to what extent can distributed PPO training accelerate policy learning for semiconductor wafer-lot dispatching while maintaining competitive operational performance?*

3 METHODOLOGY

This study investigates whether distributed PPO training can accelerate policy learning for a semiconductor-inspired digital twin environment while maintaining competitive operational performance. The methodology centers on 2D3PO as the distributed PPO framework, FabSim as the training environment, and an evaluation protocol that measures both wall-clock training time and final makespan-based performance.

3.1 Distributed PPO Framework for FabSim Training

2D3PO is the distributed training framework used in this study to accelerate PPO policy learning for FabSim by collecting simulation data in parallel across multiple compute nodes while keeping policy updates synchronized. The framework combines three components: Ray for distributed execution of FabSim environments, SB3 for the PPO implementation, and DDP for synchronized gradient updates (Moritz et al. 2018; Raffin et al. 2021; Li et al. 2020). Here, a *rollout* refers to a sequence of state, action, and reward data collected by interacting with the environment, and a *gradient* is the quantity used by the optimizer to update the policy parameters during learning. Conceptually, the design should be interpreted as synchronous data-parallel PPO: workers collect rollouts concurrently, but they follow the same policy trajectory through DDP-synchronized updates rather than training independent learners.

Figure 1 shows the overall architecture. Each node runs a local PPO training process and a set of Ray actors, where each actor manages one FabSim environment instance. During training, these actors interact with FabSim in parallel and send their collected rollouts to a node-local rollout buffer. In each transition, the policy observes the current environment state, receives a mask identifying which actions are valid, selects one valid action, and then receives the next state and shaped reward after the simulator advances. This allows multiple FabSim episodes to be collected at the same time across the cluster.

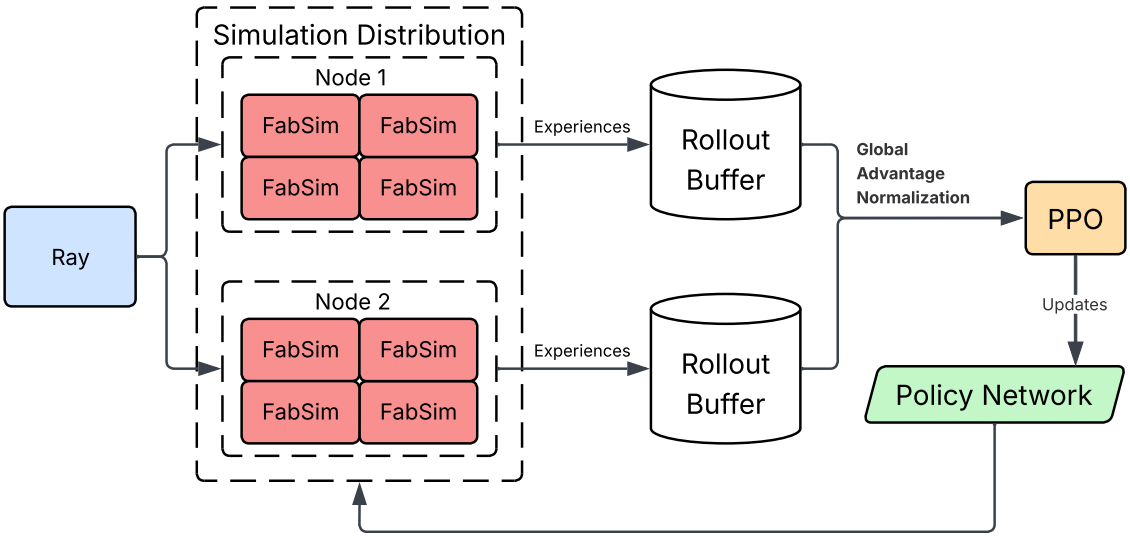

Figure 1: Distributed training architecture for FabSim policy learning. Ray actors execute FabSim environment instances in parallel across nodes and store collected transitions in per-node rollout buffers. PPO optimization is performed using synchronized updates across workers, and the resulting policy parameters are fed back into the next round of environment interaction.

After rollout collection, each node computes returns and local advantages A_i. Global normalization is then applied by aggregating the sum $s_1 = \sum_{i=1}^{n} A_i$, squared sum $s_2 = \sum_{i=1}^{n} A_i^2$, and count n across all workers. The resulting global mean μ_A, standard deviation σ_A, and normalized advantage $\tilde{A}_i$ are given in Eq. (1), where $\varepsilon > 0$ is a small numerical-stability constant. The normalized advantages are written back to each local rollout buffer before PPO optimization begins.

$$\mu_A = \frac{s_1}{n}, \qquad \sigma_A = \sqrt{\frac{s_2}{n} - \mu_A^2}, \qquad \tilde{A}_i = \frac{A_i - \mu_A}{\sigma_A + \varepsilon}. \tag{1}$$

Each node then performs PPO optimization on its own normalized rollout buffer. During backpropagation, DDP automatically combines gradients across nodes so that every copy of the policy applies the same update. In this way, rollout collection is distributed for speed, while learning remains synchronized

across the full system. The updated policy is then used for the next round of FabSim interaction, and the cycle repeats until training is complete. Algorithm 1 provides a compact summary of the overall training procedure.

Algorithm 1 Distributed PPO Training for FabSim

Require: worker processes W, rollout horizon H, training budget T

Initialize synchronized PPO policy across all workers
Launch distributed FabSim environment actors
$t \leftarrow 0$
while $t < T$ **do**
    for all workers $w \in W$ in parallel **do**
        Collect H rollout steps from local FabSim actors
        Compute local advantages
        Compute local summary statistics $(s_1^{(w)}, s_2^{(w)}, n^{(w)})$
    end for
    All-reduce $(s_1^{(w)}, s_2^{(w)}, n^{(w)})$ to obtain (s_1, s_2, n)
    Compute global advantage mean μ_A and standard deviation σ_A
    for all workers $w \in W$ in parallel **do**
        Normalize local advantages using (μ_A, σ_A)
        Update PPO with DDP-synchronized gradients
    end for
    $t \leftarrow t + |W| \cdot H$
    if evaluation checkpoint is reached **then**
        Evaluate the current policy in FabSim
        Record evaluation makespan
    end if
end while

3.2 FabSim as a Semiconductor Digital Twin Environment

FabSim is used in this work as a micro-discrete event simulation environment for training and evaluating reinforcement learning policies for semiconductor-inspired scheduling and dispatching tasks (Nsiye et al. 2024; Mondesire et al. 2025). Within the digital twin workflow considered here, FabSim provides a controllable simulation model of fab-like operations in which an RL agent repeatedly observes the current production state, selects a scheduling or dispatching action, and receives feedback based on the resulting system evolution. This repeated interaction allows the agent to learn control policies without requiring intervention in a live production setting.

At each decision point, the policy receives a state representation describing the current system condition, including relevant job, machine, and scheduling information made available by the environment. Based on this state, the policy selects an action corresponding to a wafer-lot dispatching decision within the broader scheduling problem. In the version of FabSim used in this study, invalid actions are masked out before action selection, so the policy is restricted to feasible decisions at each step. The simulator then advances, updates the production system, and returns the next state and reward signal.

During training, PPO receives a shaped reward defined as $R = -\frac{\eta_{finish}}{T_{max}}$, where η_{finish} denotes the current estimated makespan length and $T_{\max}$ is the user-defined maximum simulation time. This reward provides a normalized training signal that penalizes longer completion behavior during learning. However, policy quality is not assessed directly through this reward. Instead, the primary evaluation metric for this paper

is makespan, since it more directly reflects the operational quality of the learned dispatching policy in the tested FabSim setting. Figure 2 illustrates the FabSim decision loop used during training.

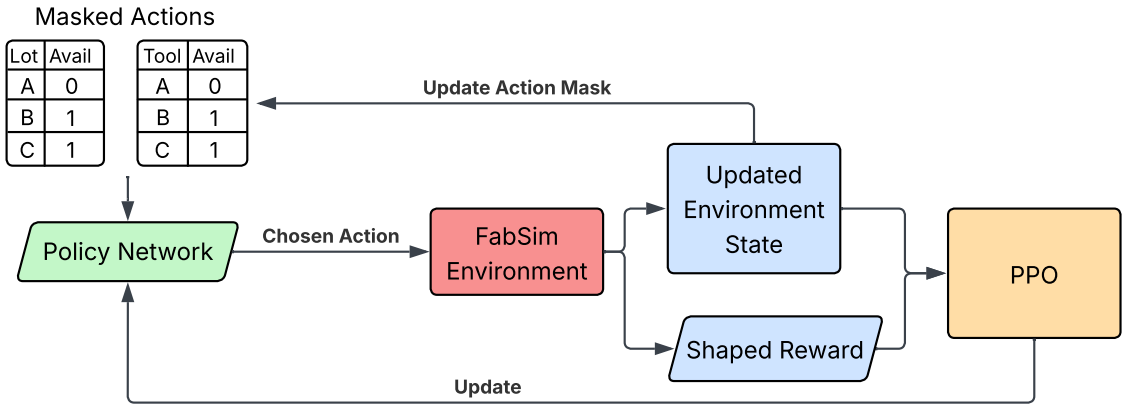

Figure 2: FabSim decision loop used during training. The current production state determines the masked feasible action set presented to the policy. After a valid scheduling or dispatching action is selected, the FabSim environment advances and returns the next state and shaped reward, which are then used by PPO for policy optimization.

3.3 Experimental Platform and Scaling Configurations

Experiments are conducted on a multi-node high-performance computing cluster managed with Slurm. The distributed training framework uses Ray v2.49.1, SB3 v2.7.0, and Torch v2.5.1. Each node is equipped with an Intel Xeon Gold 5418Y processor, 400 GB of RAM, and up to 48 CPU cores, with nodes connected by 100 Gbps HDR InfiniBand. To maintain consistent workload partitioning across configurations, at most 32 CPU cores are used per node, and each experiment is allocated exclusive access to its assigned nodes.

The evaluation considers four distributed training configurations:

- 1 node, 16 environments
- 1 node, 32 environments
- 2 nodes, 64 environments
- 4 nodes, 128 environments

A one-to-one mapping is used between environment instances and Ray actors, so increasing the configuration size directly increases the number of concurrent FabSim rollouts collected during training. These configurations are reported in the results as 16CPU, 32CPU, 64CPU, and 128CPU, respectively. Across all configurations, the policy architecture, PPO hyperparameters, total environment-step budget, and evaluation procedure are held constant; only the degree of rollout parallelism changes. This setup allows the study to examine how training runtime and learned scheduling performance change as rollout collection is scaled from a single-node baseline to larger multi-node configurations.

3.4 Training and Evaluation Protocol

Each configuration is trained for a fixed budget of 10M total environment timesteps using PPO with a consistent set of hyperparameters across experiments. This fixed-budget design allows training runtime and policy quality to be compared directly across distributed configurations because each run receives the same amount of environment interaction. Multiple independent runs are executed using different random seeds in order to capture variability in both training time and learned performance.

Policy evaluation is performed at scheduled intervals during training and reported at approximate 1 million timestep checkpoints. Since the evaluation schedule does not align exactly with uniform 1 million timestep increments, the recorded evaluation nearest each target checkpoint is used. At each evaluation point, the current policy is executed deterministically for a fixed number of episodes in FabSim, and the resulting makespan values are recorded. Final-checkpoint comparisons against the 16CPU baseline are later assessed using Dunnett tests.

It is important to distinguish between the training signal and the evaluation metric. PPO is optimized using the shaped reward returned by FabSim during interaction, while policy quality is assessed using the makespan produced during evaluation episodes. Thus, the reward is used to guide learning, whereas makespan is used to determine the practical effectiveness of the learned scheduling policy.

The two primary outcome measures are therefore:

1. **Training runtime**, measured as the total wall-clock time in minutes required to complete the full training process.
2. **Scheduling performance**, measured by the makespan achieved by the learned policy during evaluation.

Training runtime is computed as $T_{total} = T_{end} - T_{start}$.

Scheduling performance is summarized using the mean and standard deviation of makespan across evaluation runs. In the results section, these values can be compared directly across distributed configurations, with optional percentage improvement relative to the baseline configuration used to aid interpretation. The training reward is retained as part of the PPO learning process, but it is not treated as the primary reported indicator of policy quality. Together, these metrics allow the study to determine whether distributed rollout collection reduces training time while preserving useful FabSim scheduling performance.

4 RESULTS

The results of wafer-lot scheduling training within the new distributed PPO framework show a clear separation between runtime scaling and scheduling performance. Increasing the distributed configuration consistently reduced wall-clock training time, while makespan-based policy performance remained closer to the 16CPU baseline and varied more modestly over training. Figure 3, Figure 4, and Table 1 summarize these results.

4.1 Runtime Scaling

Runtime decreased as the number of environments and nodes increased with the distributed approach. As shown in Figure 3, the 32CPU configuration provided only a modest reduction relative to the 16CPU baseline, whereas the multi-node configurations produced substantially larger gains. The 64CPU configuration reduced runtime by roughly half relative to 16CPU, and 128CPU achieved the largest improvement, completing training in less than one third of the baseline runtime. Overall, the 128CPU configuration was approximately 3.7 times faster than 16CPU. The full runtime summary is reported in Table 1.

4.2 Scheduling Performance

Scheduling performance is reported as makespan ratio relative to the 16CPU baseline, where values below 1 indicate lower makespan and therefore better performance. Figure 4 shows that the 64CPU configuration remained closest to the baseline throughout training and was the only alternative configuration to finish below 1. By contrast, the 32CPU and 128CPU configurations remained above the baseline for most of training and also finished above 1. These trends are summarized in Table 1.

Final-checkpoint statistical comparisons were performed using Dunnett tests with 16CPU as the control. As reported in Table 1, none of the alternative configurations differed significantly from the baseline at

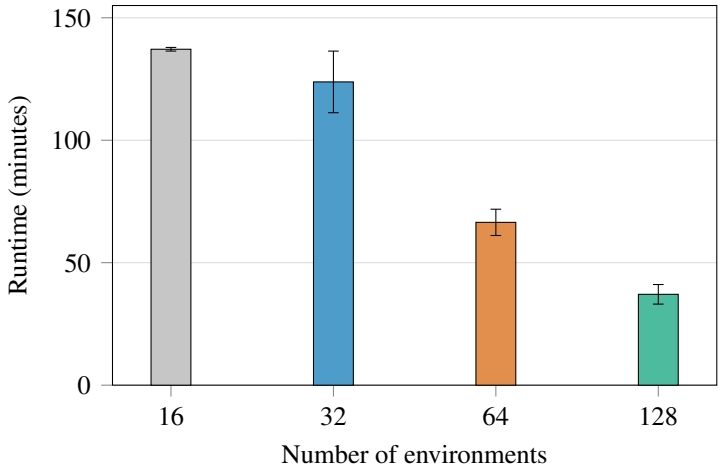

Figure 3: Total training runtime for FabSim across distributed configurations. Error bars indicate ±1 standard deviation across runs.

the final evaluation checkpoint. Thus, the runtime results showed clear separation across configurations, whereas the final makespan comparisons did not yield statistically significant differences under the test performed.

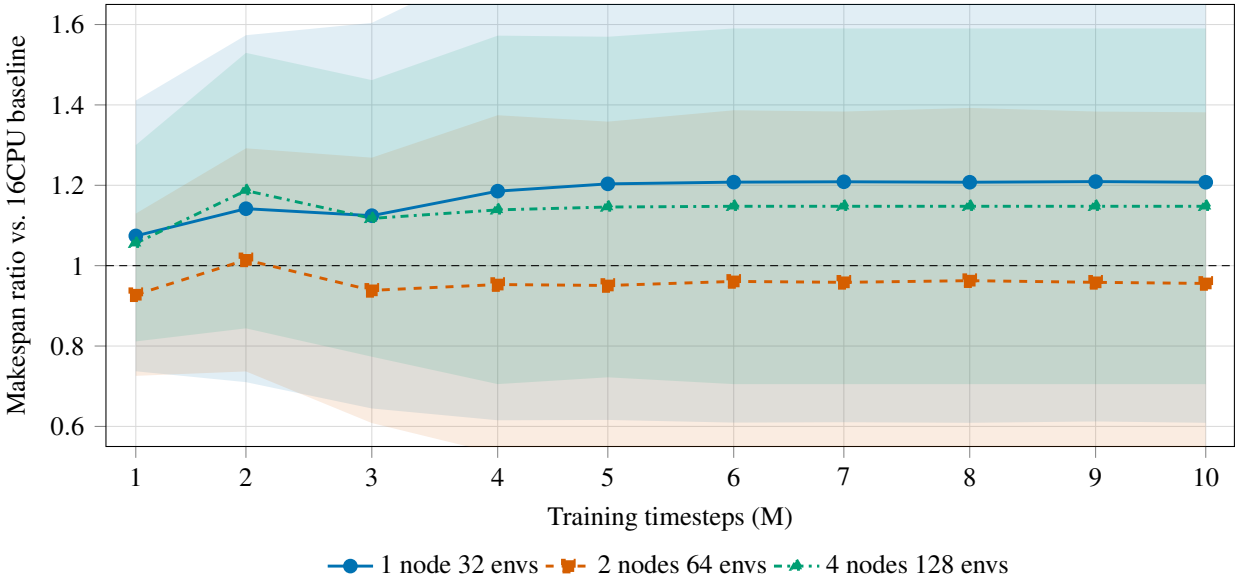

Figure 4: Makespan ratio during training relative to the 16CPU baseline. Values below 1 indicate lower makespan and therefore better performance than the baseline. Shaded bands show ±1 standard deviation across runs.

5 DISCUSSION

The results show that the most consistent effect of distributed PPO in this study was reduced wall-clock training time. Across the evaluated configurations, scaling the number of environments and nodes produced clear runtime improvements, while differences in final makespan were smaller and not statistically significant relative to the 16CPU baseline. Taken together, these results indicate that the primary value of 2D3PO

Table 1: FabSim runtime and makespan results across distributed configurations. Makespan ratio values are relative to the 16CPU baseline, where values below 1 indicate better performance. Dunnett p-values are based on final-checkpoint makespan comparisons against 16CPU.

Configuration	Runtime (min)	Makespan ratio	Dunnett p-value
16CPU	137.148 ± 0.744	1.000	—
32CPU	123.812 ± 12.579	1.177 ± 0.045	0.997
64CPU	66.473 ± 5.367	0.958 ± 0.021	0.613
128CPU	37.093 ± 3.997	1.138 ± 0.032	0.997

is as a training-systems enhancement for semiconductor digital twin workflows: it accelerates wafer-lot dispatching policy learning in FabSim under the tested conditions without a detected final-checkpoint makespan penalty relative to the baseline.

5.1 Runtime Interpretation

The runtime results show that distributed rollout collection substantially accelerated training as the configuration size increased. The largest gains occurred in the multi-node settings, with 64CPU reducing runtime by roughly half relative to the 16CPU baseline and 128CPU completing training approximately 3.7 times faster. By contrast, the 32CPU configuration yielded only a modest improvement, indicating that the strongest runtime benefits appeared once rollout collection was distributed more broadly across nodes rather than only scaled within a single smaller configuration. From a practical standpoint, 64CPU provides the most balanced runtime-performance tradeoff in this study, while 128CPU is most attractive when minimizing wall-clock training time is the dominant objective.

The observed speedups were not perfectly linear, however. A likely contributor is the evaluation procedure, in which policy evaluation is performed on the designated rank 0 process while the remaining processes wait at synchronization barriers before training resumes. As the distributed configuration grows, this evaluation-induced waiting time becomes a larger source of inefficiency and limits ideal scaling. Communication overhead from global advantage-normalization statistics and synchronized gradient updates also contributes to this deviation. Even with these effects, the runtime reductions remained substantial, showing that distributed rollout collection can significantly reduce training time for wafer-lot dispatching policy learning in the FabSim environment.

For semiconductor digital twin workflows, this runtime reduction is operationally meaningful. Faster training makes it more practical to retrain policies after changes in product mix or fab assumptions, to compare more candidate policies within the same compute budget, and to perform more iterative simulation-based what-if analyses before deployment.

5.2 Dispatching Performance Interpretation

The dispatching training performance results were more variable than the runtime results. Although Figure 4 shows differences among configurations over the course of training, these differences must be interpreted cautiously because the run-to-run variability in makespan was relatively high. Under the evaluation setting used here, different runs exposed the policy to different scheduling instances, producing a broad range of achievable makespan values across checkpoints. As a result, the ratio curves are useful for normalized comparison against the 16CPU baseline, but apparent visual separation between configurations does not by itself establish a reliable performance difference.

This interpretation is supported by the final-checkpoint statistical tests. Dunnett comparisons against the 16CPU baseline found no statistically significant differences for 32CPU, 64CPU, or 128CPU. In other words, while some configurations appeared better or worse than the baseline at different points in training,

the observed variability was large enough that these differences could not be confidently attributed to the distributed configuration itself under the tested conditions.

From the perspective of the research question, this is an important result. The study was not only concerned with reducing training time, but with doing so while maintaining useful operational performance. The absence of statistically significant final makespan differences indicates that the distributed framework reduced training time without detecting a corresponding difference in final makespan relative to the baseline in FabSim.

These findings should also be interpreted within the scope of the study. The experiments were conducted in FabSim as a micro-discrete event simulation environment, and makespan was the primary reported operational objective. Accordingly, the results provide evidence for training-system scalability in the tested HMLV semiconductor-inspired setting rather than proof of superior dispatching quality across semiconductor fabs more broadly. Even so, the results support the view that distributed PPO is a practical way to reduce the computational bottleneck that often limits RL-based digital twin experimentation.

6 CONCLUSION

This paper presented 2D3PO, a distributed PPO framework and software toolkit for wafer-lot dispatching in semiconductor manufacturing digital twins, and evaluated it in FabSim. The main contribution of this work is the design and evaluation of a practical distributed training workflow that combines Ray-based rollout collection, PyTorch Distributed Data Parallel synchronization, and Stable-Baselines3 compatibility in a single reproducible framework. This integration provides a concrete path for scaling reinforcement learning based dispatching across multiple compute nodes while retaining a widely used PPO implementation.

The results show a clear practical benefit. As the distributed configuration increased, wall-clock training time decreased substantially, with the largest evaluated configuration completing training about 3.7 times faster than the 16CPU baseline. At the same time, final makespan comparisons showed no statistically significant differences relative to the baseline under the applied Dunnett tests. Taken together, these findings show that distributed PPO can make semiconductor digital twin training markedly more computationally practical under the tested conditions.

More broadly, this study shows that distributed reinforcement learning can serve as an effective systems-level enhancement for semiconductor digital twins, where repeated policy training and scenario iteration are often constrained by simulation cost. For wafer-lot dispatching studies in particular, reducing training turnaround makes it more feasible to retrain policies, compare alternatives, and run digital twin experiments at a pace that is useful for semiconductor manufacturing research. Future work will examine more diverse fab conditions, larger problem instances, and additional operational objectives to better characterize when distributed PPO provides the strongest practical benefit.

AUTHOR BIOGRAPHIES

TO BE COMPLETED AFTER PEER-REVIEW.

REFERENCES

Agarwal, R., M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare. 2021. "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In *Advances in Neural Information Processing Systems*, Volume 34, 29304–29320 https://doi.org/10.48550/arXiv.2108.13264.

Andrychowicz, M., A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, *et al*. 2021. "What Matters for On-Policy Deep Actor-Critic Methods? A Large-Scale Study". In *International Conference on Learning Representations* https://doi.org/10.48550/arXiv.2006.05990.

Ben-Nun, T., and T. Hoefler. 2019. "Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis". *ACM Computing Surveys* 52(4) https://doi.org/10.1145/3320060.

Chung, K. T., C. K. M. Lee, and Y. P. Tsang. 2025. "Neural Combinatorial Optimization with Reinforcement Learning in Industrial Engineering: A Survey". *Artificial Intelligence Review* 58(5):130.

Engstrom, L., A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph *et al*. 2020. "Implementation Matters in Deep RL: A Case Study on PPO and TRPO". In *International Conference on Learning Representations* https://doi.org/10.48550/arXiv.2005.12729.

Garey, M. R., D. S. Johnson, and R. Sethi. 1976. "The Complexity of Flowshop and Jobshop Scheduling". *Mathematics of Operations Research* 1(2):117–129 https://doi.org/10.1287/moor.1.2.117.

Greensmith, E., P. L. Bartlett, and J. Baxter. 2004. "Variance Reduction Techniques for Gradient Estimates in Reinforcement Learning". *Journal of Machine Learning Research* 5(Nov):1471–1530.

Gupta, A. K., and A. I. Sivakumar. 2006. "Job Shop Scheduling Techniques in Semiconductor Manufacturing". *The International Journal of Advanced Manufacturing Technology* 27(11–12):1163–1169 https://doi.org/10.1007/s00170-004-2296-z.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. 2018. "Deep Reinforcement Learning that Matters". *Proceedings of the AAAI Conference on Artificial Intelligence* 32(1) https://doi.org/10.48550/arXiv.1709.06560.

Kuo, H.-A., T.-Y. Hong, and C.-F. Chien. 2025. "A Deep Reinforcement Learning Based Digital Twin Framework for Resilient Production Planning under Demand Uncertainty and an Empirical Study in Semiconductor Wafer Fabrication". *Computers & Industrial Engineering* 208:111389 https://doi.org/10.1016/j.cie.2025.111389.

Li, S., Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, *et al*. 2020. "PyTorch Distributed: Experiences on Accelerating Data Parallel Training". *Proceedings of the VLDB Endowment* 13(12):3005–3018 https://doi.org/10.14778/3415478.3415530.

Mondesire, S., M. Brown, and B. Soykan. 2025. "FabSim: A Micro-Discrete Event Simulator for Machine Learning Dynamic Job Shop Scheduling Optimizers". In *Proceedings of the 2025 Winter Simulation Conference*. Seattle, WA, USA.

Moritz, P., R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, *et al*. 2018. "Ray: A Distributed Framework for Emerging AI Applications". In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 561–577 https://doi.org/10.48550/arXiv.1712.05889.

Nsiye, E., T. Wright, B. Tse, and S. Mondesire. 2024, May. "A Micro-Discrete Event Simulation Environment for Production Scheduling in Manufacturing Digital Twins". In *Proceedings of MODSIM World*. Norfolk, Virginia, USA.

Ouahabi, N., A. Chebak, O. Kamach, O. Laayati, and M. Zegrari. 2024. "Leveraging Digital Twin into Dynamic Production Scheduling: A Review". *Robotics and Computer-Integrated Manufacturing* https://doi.org/10.1016/j.rcim.2024.102778.

Pfund, M. E., S. J. Mason, and J. W. Fowler. 2006. "Semiconductor Manufacturing Scheduling and Dispatching: State of the Art and Survey of Needs". In *Handbook of Production Scheduling*, 213–241. Springer https://doi.org/10.1007/0-387-33117-4_9.

Raffin, A., A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. 2021. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". *Journal of Machine Learning Research* 22(268):1–8.

Sarin, S. C., A. Varadarajan, and L. Wang. 2011. "A Survey of Dispatching Rules for Operational Control in Wafer Fabrication". *Production Planning & Control* 22(1):4–24 https://doi.org/10.1080/09537287.2010.490014.

Schulman, J., P. Moritz, S. Levine, M. Jordan, and P. Abbeel. 2015. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". *arXiv preprint arXiv:1506.02438* https://doi.org/10.48550/arXiv.1506.02438.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. 2017. "Proximal Policy Optimization Algorithms". *arXiv preprint arXiv:1707.06347* https://doi.org/10.48550/arXiv.1707.06347.

Schwede, C., and D. Fischer. 2024. "Learning Simulation-Based Digital Twins for Discrete Material Flow Systems: A Review". In *2024 Winter Simulation Conference (WSC)*, 3070–3081: IEEE https://doi.org/10.1109/WSC63780.2024.10838729.

Shallue, C. J., J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl. 2019. "Measuring the Effects of Data Parallelism on Neural Network Training". *Journal of Machine Learning Research* 20(112):1–49 https://doi.org/10.48550/arXiv.1811.03600.

Stöckermann, P., S. Feudel, A. Immordino, N. Hayen, T. Altenmüller, M. Gebser, *et al*. 2025. "Reinforcement Learning Based Dispatching Solutions in Semiconductor Manufacturing: A Literature Review on Validation and Deployment". *Production & Manufacturing Research* 13(1) https://doi.org/10.1080/21693277.2025.2582472.

Sutton, R. S., D. McAllester, S. Singh, and Y. Mansour. 1999. "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In *Advances in Neural Information Processing Systems*, Volume 12.

Tassel, P., B. Kovács, M. Gebser, K. Schekotihin, P. Stöckermann, and G. Seidel. 2023. "Semiconductor Fab Scheduling with Self-Supervised and Reinforcement Learning". In *2023 Winter Simulation Conference (WSC)*, 1924–1935 https://doi.org/10.1109/WSC60868.2023.10407747.

Towers, M., A. Kwiatkowski, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, *et al*. 2025. "Gymnasium: A Standard Interface for Reinforcement Learning Environments". In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* https://doi.org/10.48550/arXiv.2407.17032.

Waschneck, B., A. Reichstaller, L. Belzner, T. Altenmüller, T. Bauernhansl, A. Knapp *et al*. 2018. "Optimization of Global Production Scheduling with Deep Reinforcement Learning". *Procedia CIRP* 72:1264–1269 https://doi.org/10.1016/j.procir.2018.03.212.

Wooley, A., J. Bitencourt, and D. F. Silva. 2025. "Bridging the Gap between Discrete Event Simulation and Digital Twin: A Manufacturing Case Study". *Manufacturing Letters* 44:1274–1284 https://doi.org/10.1016/j.mfglet.2025.06.147.

Yedidsion, H., D. Norman, P. Dawadi, D. Adams, L. Krebs, and E. Zarifoglu. 2025. "Productization of Reinforcement Learning in Semiconductor Manufacturing". In *Proceedings of the 2025 Winter Simulation Conference*.